

Concurrency: Deadlock and Starvation

Chapter 6

Agenda:

- Two problems that affects all efforts to support concurrent processing:
 - Deadlock and Starvation
- Principles of deadlock
- Related problem of starvation
- Common approaches to dealing with deadlock
 - Prevention ,Detection, Avoidance
- Classic problems used to illustrate both synchronization and deadlock issues:
 - The dining philosophers problem.
- **Note:** Consideration of concurrency and deadlock on a **single** system.

Deadlock

- What???
- Permanent blocking of a set of processes that either compete for system resources or communicate with each other.
- Deadlock is permanent.
- What to do when
 - No efficient solution
 - Deadlocks involve Involve conflicting needs for resources by two or more processes

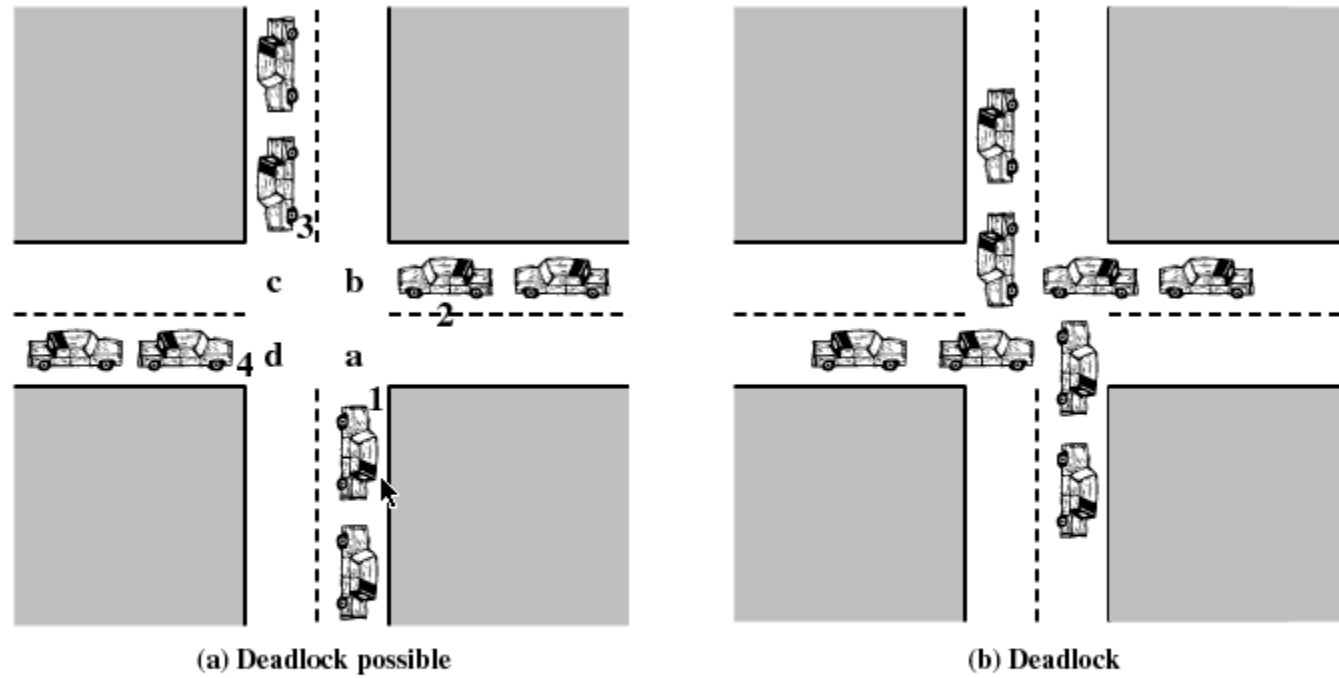


Figure 6.1 Illustration of Deadlock

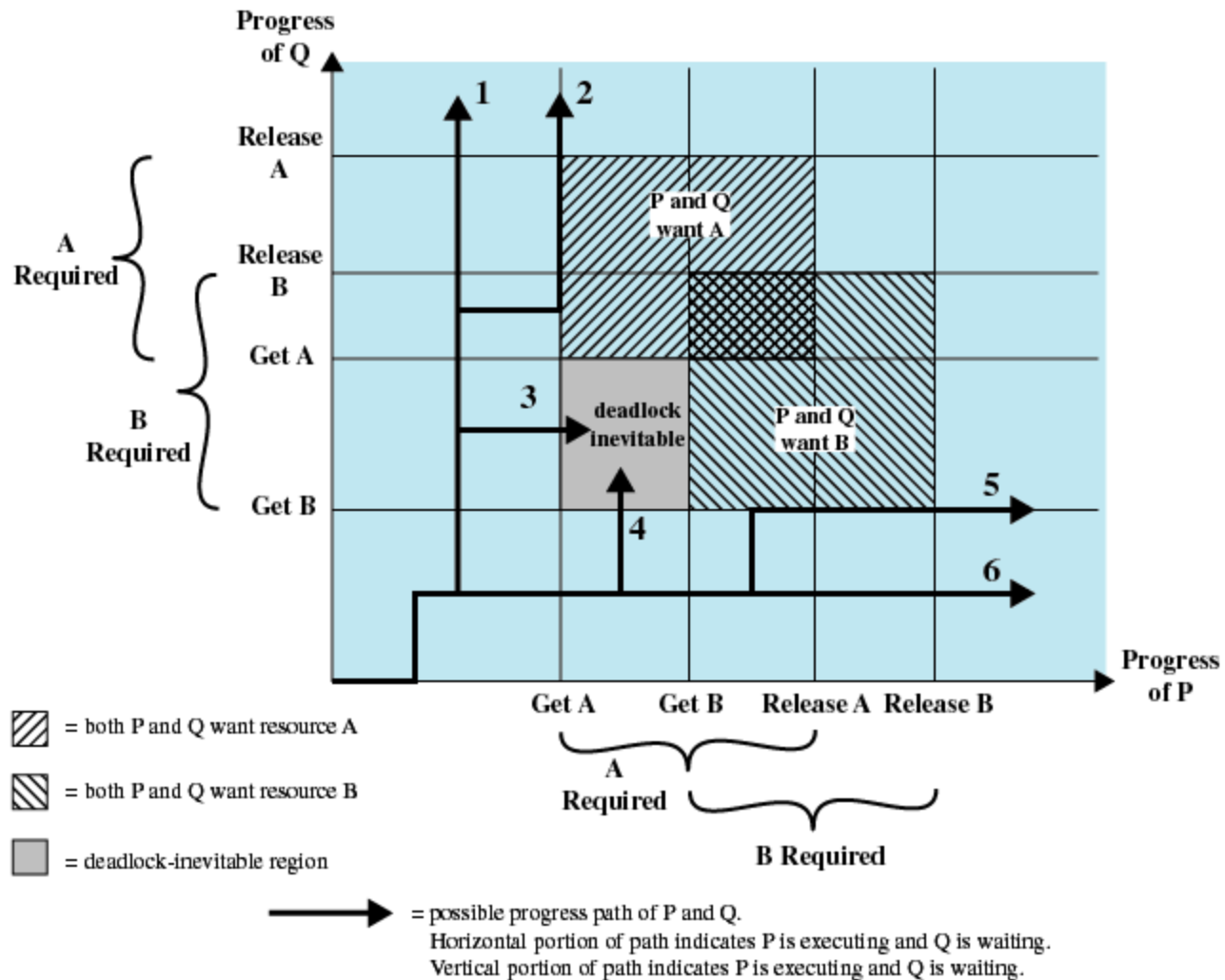


Figure 6.2 Example of Deadlock

- Different execution paths:
- 1. Q acquires B and then A and then releases B and A. When P resumes execution, it will be able to acquire both resources.
- 2. Q acquires B and then A. P executes and blocks on a request for A. Q releases B and A. When P resumes execution, it will be able to acquire both resources.
- 3. Q acquires B and then P acquires A. Deadlock is inevitable, because as execution proceeds, Q will block on A and P will block on B.
- 4. P acquires A and then Q acquires B. Deadlock is inevitable, because as execution proceeds, Q will block on A and P will block on B.
- 5. P acquires A and then B. Q executes and blocks on a request for B. P releases A and B. When Q resumes execution, it will be able to acquire both resources.
- 6. P acquires A and then B and then releases A and B. When Q resumes execution, it will be able to acquire both resources.

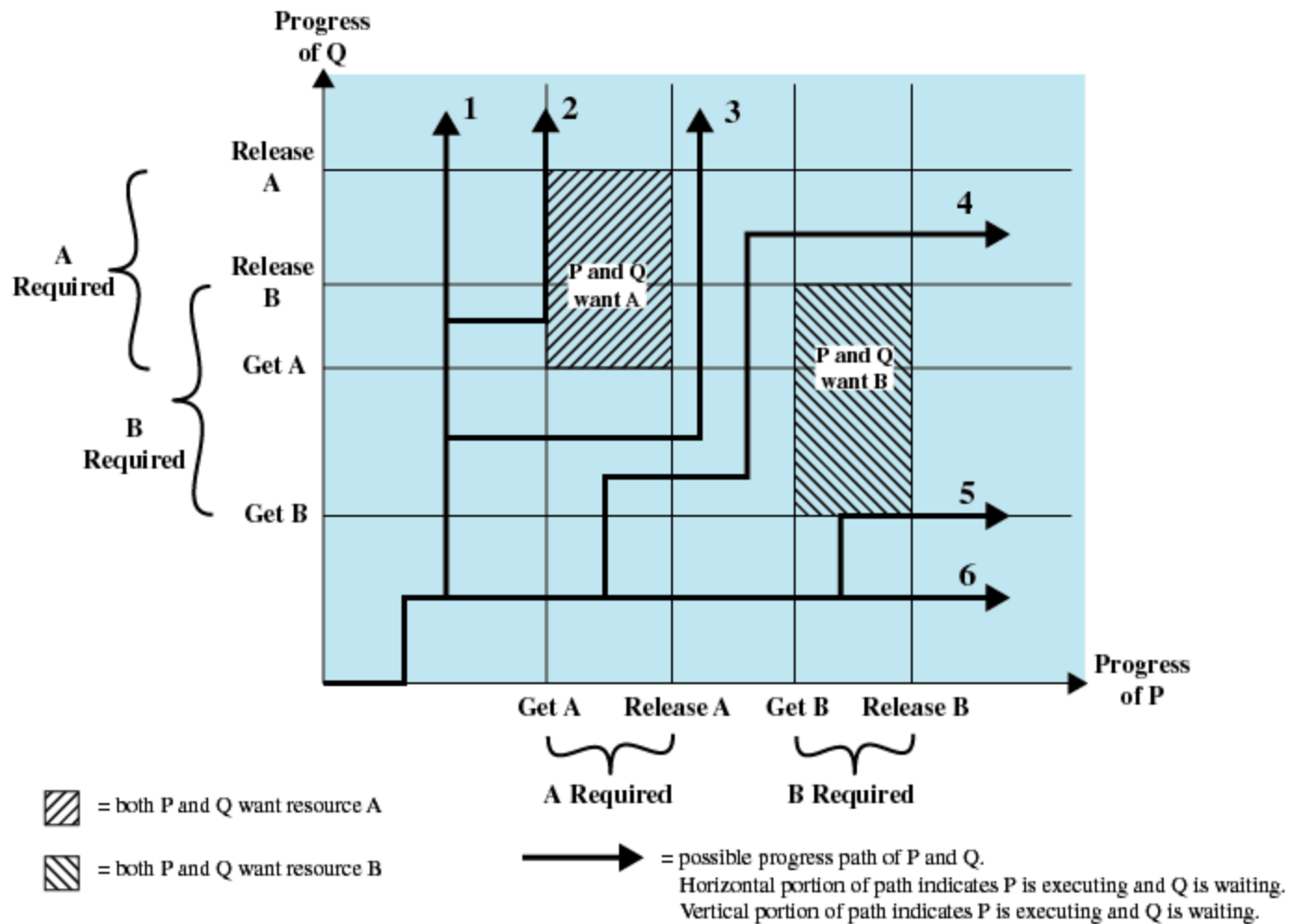


Figure 6.3 Example of No Deadlock [BACO03]

Reusable Resources

- Used by only one process at a time and not depleted by that use
- Processes obtain resources that they later release for reuse by other processes
- Processors, I/O channels, main and secondary memory, devices, and data structures such as files, databases, and semaphores
- Deadlock occurs if each process holds one resource and requests the other

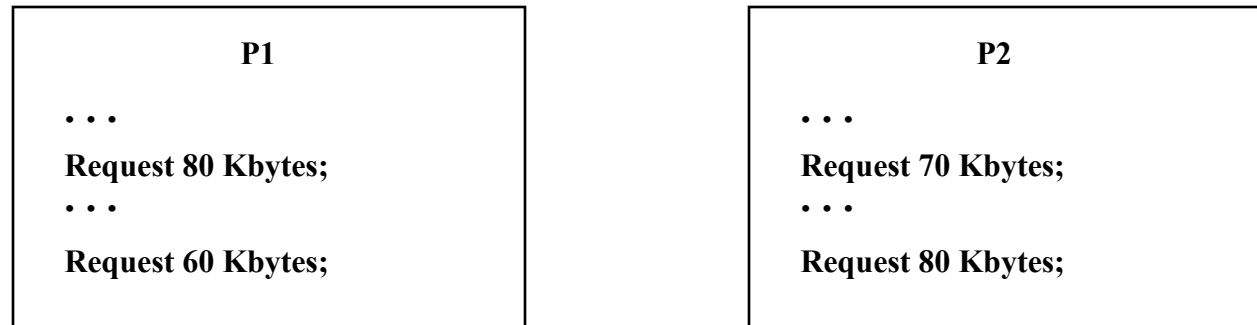
Example of Deadlock

Process P		Process Q	
Step	Action	Step	Action
p ₀	Request (D)	q ₀	Request (T)
p ₁	Lock (D)	q ₁	Lock (T)
p ₂	Request (T)	q ₂	Request (D)
p ₃	Lock (T)	q ₃	Lock (D)
p ₄	Perform function	q ₄	Perform function
p ₅	Unlock (D)	q ₅	Unlock (T)
p ₆	Unlock (T)	q ₆	Unlock (D)

Figure 6.4 Example of Two Processes Competing for Reusable Resources

Another Example of Deadlock

- Space is available for allocation of 200Kbytes, and the following sequence of events occur



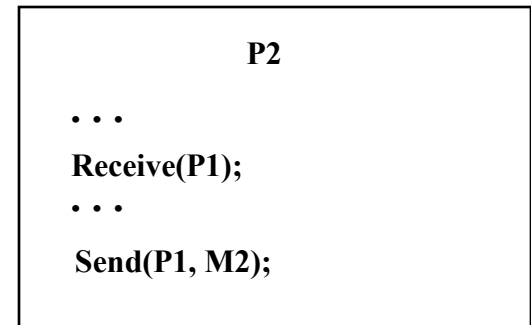
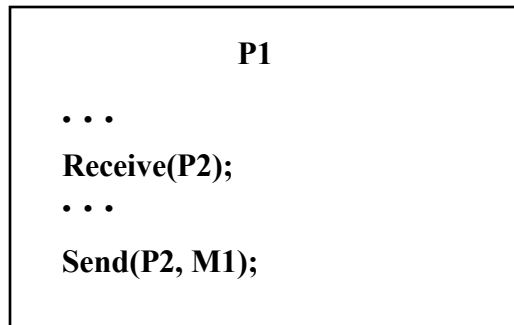
- Deadlock occurs if both processes progress to their second request

Consumable Resources

- Created (produced) and destroyed (consumed)
- Interrupts, signals, messages, and information in I/O buffers
- Deadlock may occur if a Receive message is blocking
- May take a rare combination of events to cause deadlock

Example of Deadlock

- Deadlock occurs if receive is blocking

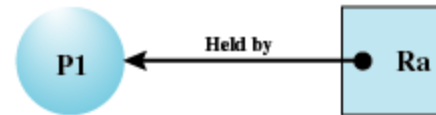


Resource Allocation Graphs

- Directed graph that depicts a state of the system of resources and processes

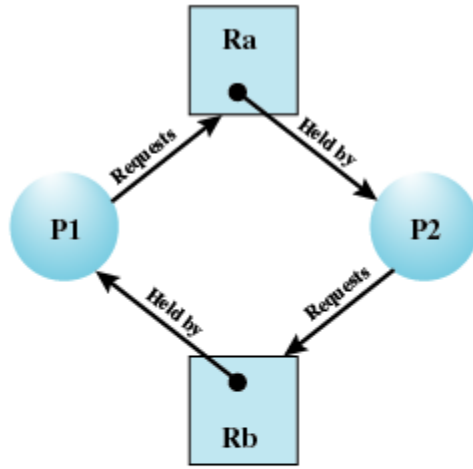


(a) Resource is requested

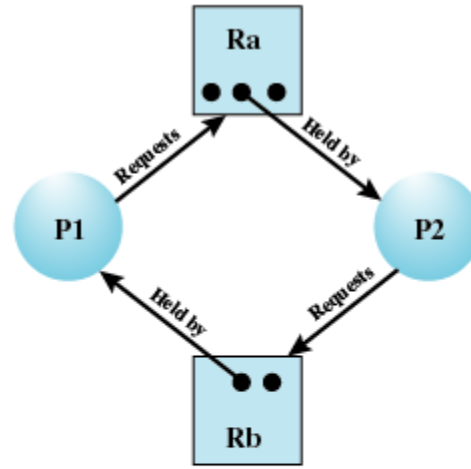


(b) Resource is held

Resource Allocation Graphs



(c) Circular wait



(d) No deadlock

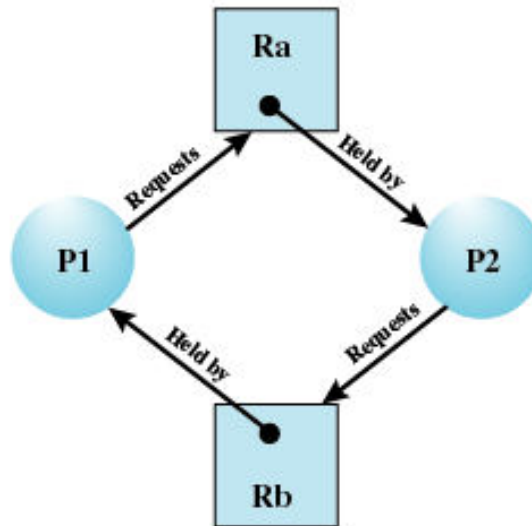
Figure 6.5 Examples of Resource Allocation Graphs

Conditions for Deadlock

- When????
- Mutual exclusion
 - Only one process may use a resource at a time
- Hold-and-wait
 - A process may hold allocated resources while awaiting assignment of others
- No preemption
 - No resource can be forcibly removed from a process holding it

Conditions for Deadlock

- Circular wait
 - A closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain



(c) Circular wait

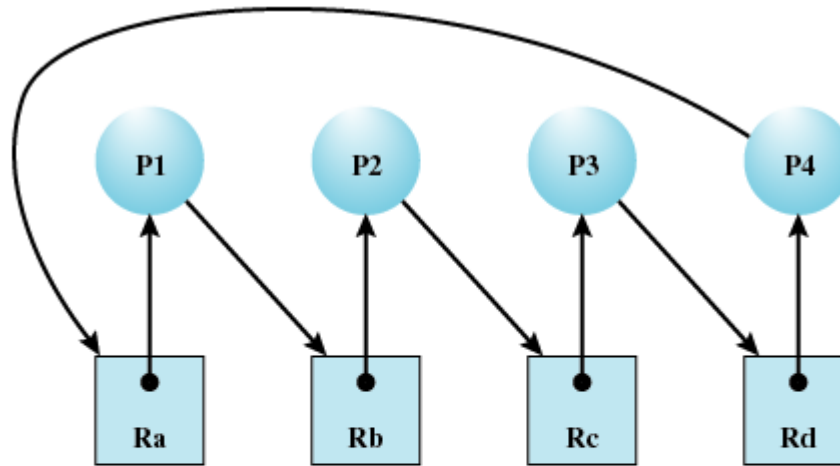


Figure 6.6 Resource Allocation Graph for Figure 6.1b

Possibility of Deadlock

- Mutual Exclusion
- No preemption
- Hold and wait

Existence of Deadlock

- Mutual Exclusion
- No preemption
- Hold and wait
- Circular wait

Deadlock Prevention

- Mutual Exclusion
 - Must be supported by the operating system
- Hold and Wait
 - Require a process request all of its required resources at one time

Deadlock Prevention

- Prob:
 - Process may be held up for a long time waiting for all of its resource requests to be filled, when in fact it could have proceeded with only some of the resources.
 - Resources allocated to a process may remain unused for a considerable period, during which time they are denied to other processes.
 - A process may not know in advance all of the resources that it will require.

Deadlock Prevention

- No Preemption
 - Process must release resource and request again
 - Operating system may preempt a process to require it releases its resources
- Circular Wait
 - Define a linear ordering of resource types

Deadlock Avoidance

- A decision is made dynamically whether the current resource allocation request will, if granted, potentially lead to a deadlock
- Requires knowledge of future process request

Two Approaches to Deadlock Avoidance

- Do not start a process if its demands might lead to deadlock
- Do not grant an incremental resource request to a process if this allocation might lead to deadlock

Resource Allocation Denial

- Referred to as the banker's algorithm
- **State** of the system is the current allocation of resources to process
- **Safe state** is where there is at least one sequence that does not result in deadlock
- **Unsafe state** is a state that is not safe

Resource Allocation Denial

Resource = $\mathbf{R} = (R_1, R_2, \dots, R_m)$	Total amount of each resource in the system
Available = $\mathbf{V} = (V_1, V_2, \dots, V_m)$	Total amount of each resource not allocated to any process
Claim = $\mathbf{C} = \begin{pmatrix} C_{11} & C_{12} & \dots & C_{1m} \\ C_{21} & C_{22} & \dots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \dots & C_{nm} \end{pmatrix}$	C_{ij} = requirement of process i for resource j
Allocation = $\mathbf{A} = \begin{pmatrix} A_{11} & A_{12} & \dots & A_{1m} \\ A_{21} & A_{22} & \dots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \dots & A_{nm} \end{pmatrix}$	A_{ij} = current allocation to process i of resource j

Determination of a Safe State

Initial State

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	3	2	2	P1	1	0	0	P1	2	2	2
P2	6	1	3	P2	6	1	2	P2	0	0	1
P3	3	1	4	P3	2	1	1	P3	1	0	3
P4	4	2	2	P4	0	0	2	P4	4	2	0
Claim matrix C				Allocation matrix A				C - A			
	R1	R2	R3		R1	R2	R3				
Resource vector R				Available vector V							
9 3 6				0 1 1							

(a) Initial state

- Can any of the four processes be run to completion with the resources available?

Determination of a Safe State

P2 Runs to Completion

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
6	2	3

Available vector **V**

(b) P2 runs to completion

Determination of a Safe State

P1 Runs to Completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
7	2	3

Available vector **V**

(c) P1 runs to completion

Determination of a Safe State

P3 Runs to Completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
9	3	4

Available vector V

(d) P3 runs to completion

following deadlock avoidance strategy,
ensures that the system of processes and
resources is always in a **safe state**.

Determination of an Unsafe State

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	1	0	2
P3	1	0	3
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
1	1	2

Available vector **V**

(a) Initial state

Determination of an Unsafe State

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	1	2	1
P2	1	0	2
P3	1	0	3
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

(b) P1 requests one unit each of R1 and R3

- The deadlock avoidance strategy does not predict deadlock with certainty;
- It merely anticipates the possibility of deadlock and assures that there is never such a possibility

Deadlock Avoidance Logic

```
struct state
{
    int resource[m];
    int available[m];
    int claim[n][m];
    int alloc[n][m];
}
```

(a) global data structures

```
if (alloc [i,*] + request [*] > claim [i,*])
    < error >; /* total request > claim*/
else if (request [*] > available [*])
    < suspend process >;
else /* simulate alloc */
{
    < define newstate by:
    alloc [i,*] = alloc [i,*] + request [*];
    available [*] = available [*] - request [*] >;
}
if (safe (newstate))
    < carry out allocation >;
else
{
    < restore original state >;
    < suspend process >;
}
```

(b) resource alloc algorithm

Deadlock Avoidance Logic

```
boolean safe (state S)
{
    int currentavail[m];
    process rest[<number of processes>];
    currentavail = available;
    rest = {all processes};
    possible = true;
    while (possible)
    {
        <find a process  $P_k$  in rest such that
            claim  $[k,*] - \text{alloc } [k,*] \leq \text{currentavail};>$ 
        if (found)                                /* simulate execution of  $P_k$  */
        {
            currentavail = currentavail + alloc  $[k,*]$ ;
            rest = rest -  $\{P_k\}$ ;
        }
        else
            possible = false;
    }
    return (rest == null);
}
```

(c) test for safety algorithm (banker's algorithm)

Deadlock Avoidance

- Adv.
 - Less restrictive than prevention strategy.
- Limitations:
 - Maximum resource requirement must be stated in advance
 - Processes under consideration must be independent; no synchronization requirements
 - There must be a fixed number of resources to allocate
 - No process may exit while holding resources

Deadlock Detection

- Deadlock prevention strategies are conservative
- They solve the problem of deadlock by **limiting access to resources and by imposing restrictions on processes.**
- Deadlock detection strategies do not limit resource
- access or restrict process actions.
- Deadlock detection, **requested resources are granted to processes whenever possible.**
- Periodically, the OS performs an algorithm that allows it to detect the circular wait condition described earlier in condition.

Deadlock Detection

- Frequency of checking????
 - each resource request **OR**
 - depending on how likely it is for a deadlock to occur.
- Adv 1st
 - It leads to early detection,
 - The algorithm is relatively simple because it is based on incremental changes to the state of the system.
- Disadv:
 - frequent checks consume considerable processor time.

Deadlock Detection

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A

	R1	R2	R3	R4	R5
	2	1	1	2	1

Resource vector

	R1	R2	R3	R4	R5
	0	0	0	0	1

Available vector

Figure 6.10 Example for Deadlock Detection

- The algorithm proceeds by marking processes that are **not deadlocked**.
- Initially, all processes are unmarked.

Deadlock Detection Algorithm

1. Mark each process that has a row in the allocation matrix of all zeros
2. Initialize a temporary vector W to equal to available vector.
3. Find an index i such that process i is currently unmarked and the i th row of request matrix is less than or equal to W .
 - **If no such row is found, terminate the algorithm and all unmarked processes are those involved in the deadlock.**
4. If such a row is found, mark process i and add the corresponding row of the allocation matrix to W . Return to step 3

Deadlock Detection Algorithm

- A deadlock exists **if and only if there are unmarked processes at the end of the algorithm.**
- Each unmarked process is deadlocked.
- Algorithm does not guarantee to prevent deadlock
- All that it does is determine if deadlock currently exists.

Strategies once Deadlock Detected

- Abort all deadlocked processes
- Back up each deadlocked process to some previously defined checkpoint, and restart all process
 - Original deadlock may occur
- Successively abort deadlocked processes until deadlock no longer exists
- Successively preempt resources until deadlock no longer exists

Selection Criteria Deadlocked Processes

- Least amount of processor time consumed so far
- Least number of lines of output produced so far
- Most estimated time remaining
- Least total resources allocated so far
- Lowest priority

Strengths and Weaknesses of the Strategies

Table 6.1 Summary of Deadlock Detection, Prevention, and Avoidance Approaches for Operating Systems [ISLO80]

Approach	Resource Allocation Policy	Different Schemes	Major Advantages	Major Disadvantages
Prevention	Conservative; undercommits resources	Requesting all resources at once	• Works well for processes that perform a single burst of activity • No preemption necessary	• Inefficient • Delays process initiation • Future resource requirements must be known by processes
		Preemption	• Convenient when applied to resources whose state can be saved and restored easily	• Preempts more often than necessary
		Resource ordering	• Feasible to enforce via compile-time checks • Needs no run-time computation since problem is solved in system design	• Disallows incremental resource requests
Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	• No preemption necessary	• Future resource requirements must be known by OS • Processes can be blocked for long periods
Detection	Very liberal; requested resources are granted where possible	Invoke periodically to test for deadlock	• Never delays process initiation • Facilitates on-line handling	• Inherent preemption losses

Dining Philosophers Problem

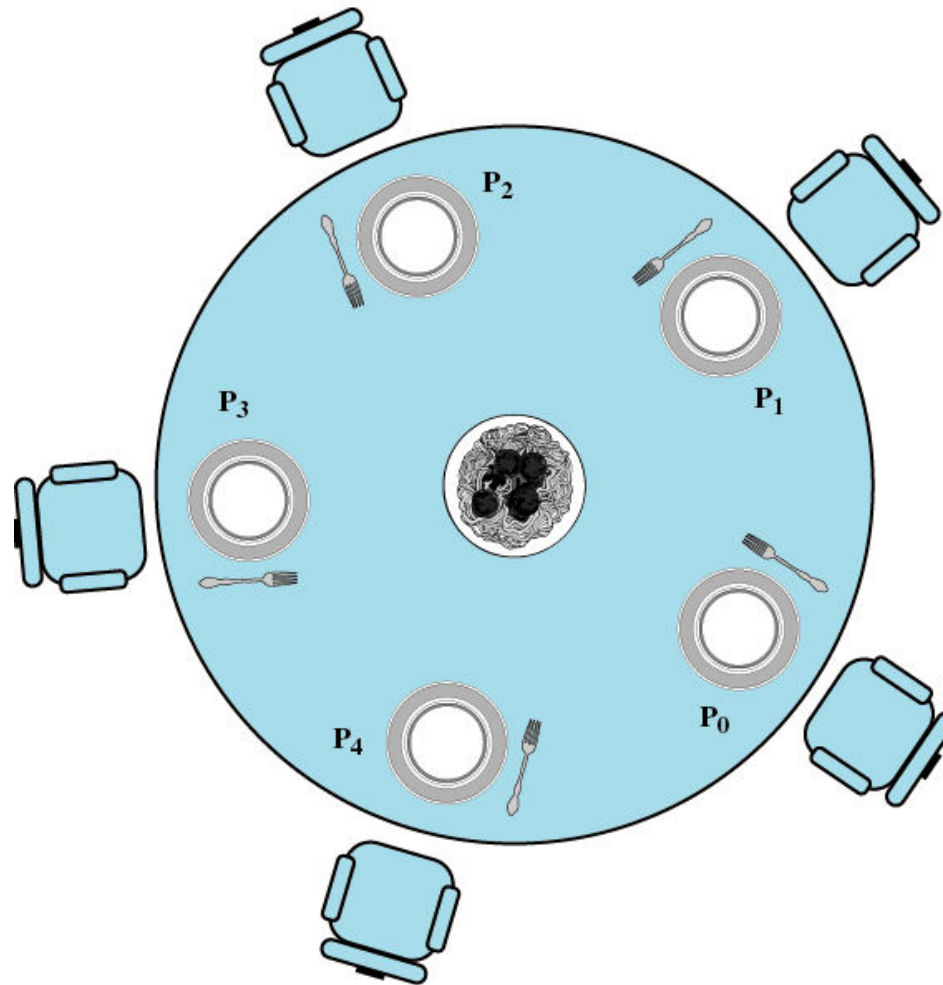


Figure 6.11 Dining Arrangement for Philosophers

Dining Philosophers Problem

```
/* program      diningphilosophers */
semaphore fork [5] = {1};
int i;
void philosopher (int i)
{
    while (true)
    {
        think();
        wait (fork[i]);
        wait (fork [(i+1) mod 5]);
        eat();
        signal(fork [(i+1) mod 5]);
        signal(fork[i]);
    }
}
void main()
{
    parbegin (philosopher (0), philosopher (1), philosopher (2),
              philosopher (3), philosopher (4));
}
```

Figure 6.12 A First Solution to the Dining Philosophers Problem

Dining Philosophers Problem

```
/* program diningphilosophers */
semaphore fork[5] = {1};
semaphore room = {4};
int i;
void philosopher (int I)
{
    while (true)
    {
        think();
        wait (room);
        wait (fork[i]);
        wait (fork [(i+1) mod 5]);
        eat();
        signal (fork [(i+1) mod 5]);
        signal (fork[i]);
        signal (room);
    }
}
void main()
{
    parbegin (philosopher (0), philosopher (1), philosopher (2),
              philosopher (3), philosopher (4));
}
```

Figure 6.13 A Second Solution to the Dining Philosophers Problem

Dining Philosophers Problem

```
monitor dining_controller;
cond ForkReady[5]; /* condition variable for synchronization */
boolean fork[5] = {true}; /* availability status of each fork */

void get_forks(int pid) /* pid is the philosopher id number */
{
    int left = pid;
    int right = (pid++) % 5;
    /*grant the left fork*/
    if (!fork(left))
        cwait(ForkReady[left]); /* queue on condition variable */
    fork(left) = false;
    /*grant the right fork*/
    if (!fork(right))
        cwait(ForkReady[right]); /* queue on condition variable */
    fork(right) = false;
}

void release_forks(int pid)
{
    int left = pid;
    int right = (pid++) % 5;
    /*release the left fork*/
    if (empty(ForkReady[left]) /*no one is waiting for this fork */
        fork(left) = true;
    else /* awaken a process waiting on this fork */
        csignal(ForkReady[left]);
    /*release the right fork*/
    if (empty(ForkReady[right]) /*no one is waiting for this fork */
        fork(right) = true;
    else /* awaken a process waiting on this fork */
        csignal(ForkReady[right]);
}
```

```
void philosopher[k=0 to 4] /* the five philosopher clients */
{
    while (true)
    {
        <think>;
        get_forks(k); /* client requests two forks via monitor */
        <eat spaghetti>;
        release_forks(k); /* client releases forks via the monitor */
    }
}
```

Figure 6.14 A Solution to the Dining Philosophers Problem Using a Monitor

Dining Philosophers Problem

```
monitor dining_controller;
enum states {thinking, hungry, eating} state[5];
cond needFork[5] /* condition variable */

void get_forks(int pid) /* pid is the philosopher id number */
{
    state[pid] = hungry; /* announce that I'm hungry */
    if (state[(pid+1) % 5] == eating
        || (state[(pid-1) % 5] == eating
            cwait(needFork[pid]); /* wait if either neighbor is eating */
        state[pid] = eating; /* proceed if neither neighbor is eating */
    }

void release_forks(int pid)
{
    state[pid] = thinking;
    /* give right (higher) neighbor a chance to eat */
    if (state[(pid+1) % 5] == hungry
        || (state[(pid+2) % 5] != eating)
        csignal(needFork[pid+1]);
    /* give left (lower) neighbor a chance to eat */
    else if (state[(pid-1) % 5] == hungry
        || (state[(pid-2) % 5] != eating)
        csignal(needFork[pid-1]);
    }
}
```

```
void philosopher[k=0 to 4] /* the five philosopher clients */
{
    while (true)
    {
        <think>;
        get_forks(k); /* client requests two forks via monitor */
        <eat spaghetti>;
        release_forks(k); /* client releases forks via the monitor */
    }
}
```

Exercise

Given the following state for the Banker's Algorithm.

6 processes P0 through P5

4 resource types: A (15 instances); B (6 instances)

C (9 instances); D (10 instances)

Snapshot at time T0:

Available								
A		B		C		D		
6		3		5		4		
Process	Current allocation				Maximum demand			
	A	B	C	D	A	B	C	D
P0	2	0	2	1	9	5	5	5
P1	0	1	1	1	2	2	3	3
P2	4	1	0	2	7	5	4	4
P3	1	0	0	1	3	3	3	2
P4	1	1	0	0	5	2	2	1
P5	1	0	1	1	4	4	4	4

- Verify that the Available array has been calculated correctly.
- Calculate the Need matrix.
- Show that the current state is safe, that is, show a safe sequence of processes. In addition, to the sequence show how the Available (working array) changes as each process terminates.
- Given the request (3,2,3,3) from Process P5. Should this request be granted? Why or why not?

UNIX Concurrency Mechanisms

- Pipes
- Messages
- Shared memory
- Semaphores
- Signals

Table 6.2 UNIX Signals

Value	Name	Description
01	SIGHUP	Hang up; sent to process when kernel assumes that the user of that process is doing no useful work
02	SIGINT	Interrupt
03	SIGQUIT	Quit; sent by user to induce halting of process and production of core dump
04	SIGILL	Illegal instruction
05	SIGTRAP	Trace trap; triggers the execution of code for process tracing
06	SIGIOT	IOT instruction
07	SIGEMT	EMT instruction
08	SIGFPE	Floating-point exception
09	SIGKILL	Kill; terminate process
10	SIGBUS	Bus error
11	SIGSEGV	Segmentation violation; process attempts to access location outside its virtual address space
12	SIGSYS	Bad argument to system call
13	SIGPIPE	Write on a pipe that has no readers attached to it
14	SIGALRM	Alarm clock; issued when a process wishes to receive a signal after a period of time
15	SIGTERM	Software termination
16	SIGUSR1	User-defined signal 1
17	SIGUSR2	User-defined signal 2
18	SIGCHLD	Death of a child
19	SIGPWR	Power failure

Linux Kernel Concurrency Mechanisms

- Includes all the mechanisms found in UNIX
- Atomic operations execute without interruption and without interference

Linux Atomic Operations

Atomic Integer Operations	
<code>ATOMIC_INIT (int i)</code>	At declaration: initialize an <code>atomic_t</code> to <code>i</code>
<code>int atomic_read(atomic_t *v)</code>	Read integer value of <code>v</code>
<code>void atomic_set(atomic_t *v, int i)</code>	Set the value of <code>v</code> to integer <code>i</code>
<code>void atomic_add(int i, atomic_t *v)</code>	Add <code>i</code> to <code>v</code>
<code>void atomic_sub(int i, atomic_t *v)</code>	Subtract <code>i</code> from <code>v</code>
<code>void atomic_inc(atomic_t *v)</code>	Add 1 to <code>v</code>
<code>void atomic_dec(atomic_t *v)</code>	Subtract 1 from <code>v</code>
<code>int atomic_sub_and_test(int i, atomic_t *v)</code>	Subtract <code>i</code> from <code>v</code> ; return 1 if the result is zero; return 0 otherwise
<code>int atomic_add_negative(int i, atomic_t *v)</code>	Add <code>i</code> to <code>v</code> ; return 1 if the result is negative; return 0 otherwise (used for implementing semaphores)
<code>int atomic_dec_and_test(atomic_t *v)</code>	Subtract 1 from <code>v</code> ; return 1 if the result is zero; return 0 otherwise
<code>int atomic_inc_and_test(atomic_t *v)</code>	Add 1 to <code>v</code> ; return 1 if the result is zero; return 0 otherwise

Linux Atomic Operations

Atomic Bitmap Operations	
<code>void set_bit(int nr, void *addr)</code>	Set bit <code>nr</code> in the bitmap pointed to by <code>addr</code>
<code>void clear_bit(int nr, void *addr)</code>	Clear bit <code>nr</code> in the bitmap pointed to by <code>addr</code>
<code>void change_bit(int nr, void *addr)</code>	Invert bit <code>nr</code> in the bitmap pointed to by <code>addr</code>
<code>int test_and_set_bit(int nr, void *addr)</code>	Set bit <code>nr</code> in the bitmap pointed to by <code>addr</code> ; return the old bit value
<code>int test_and_clear_bit(int nr, void *addr)</code>	Clear bit <code>nr</code> in the bitmap pointed to by <code>addr</code> ; return the old bit value
<code>int test_and_change_bit(int nr, void *addr)</code>	Invert bit <code>nr</code> in the bitmap pointed to by <code>addr</code> ; return the old bit value
<code>int test_bit(int nr, void *addr)</code>	Return the value of bit <code>nr</code> in the bitmap pointed to by <code>addr</code>

Linux Kernel Concurrency Mechanisms

- Spinlocks
 - Used for protecting a critical section

Table 6.4 Linux Spinlocks

<code>void spin_lock(spinlock_t *lock)</code>	Acquires the specified lock, spinning if needed until it is available
<code>void spin_lock_irq(spinlock_t *lock)</code>	Like <code>spin_lock</code> , but also disables interrupts on the local processor
<code>void spin_lock_irqsave(spinlock_t *lock, unsigned long flags)</code>	Like <code>spin_lock_irq</code> , but also saves the current interrupt state in flags
<code>void spin_lock_bh(spinlock_t *lock)</code>	Like <code>spin_lock</code> , but also disables the execution of all bottom halves
<code>void spin_unlock(spinlock_t *lock)</code>	Releases given lock
<code>void spin_unlock_irq(spinlock_t *lock)</code>	Releases given lock and enables local interrupts
<code>void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags)</code>	Releases given lock and restores local interrupts to given previous state
<code>void spin_unlock_bh(spinlock_t *lock)</code>	Releases given lock and enables bottom halves
<code>void spin_lock_init(spinlock_t *lock)</code>	Initializes given spinlock
<code>int spin_trylock(spinlock_t *lock)</code>	Tries to acquire specified lock; returns nonzero if lock is currently held and zero otherwise
<code>int spin_is_locked(spinlock_t *lock)</code>	Returns nonzero if lock is currently held and zero otherwise

Table 6.5 Linux Semaphores

Traditional Semaphores	
<code>void sema_init(struct semaphore *sem, int count)</code>	Initializes the dynamically created semaphore to the given count
<code>void init_MUTEX(struct semaphore *sem)</code>	Initializes the dynamically created semaphore with a count of 1 (initially unlocked)
<code>void init_MUTEX_LOCKED(struct semaphore *sem)</code>	Initializes the dynamically created semaphore with a count of 0 (initially locked)
<code>void down(struct semaphore *sem)</code>	Attempts to acquire the given semaphore, entering uninterruptible sleep if semaphore is unavailable
<code>int down_interruptible(struct semaphore *sem)</code>	Attempts to acquire the given semaphore, entering interruptible sleep if semaphore is unavailable; returns <code>-EINTR</code> value if a signal other than the result of an up operation is received.
<code>int down_trylock(struct semaphore *sem)</code>	Attempts to acquire the given semaphore, and returns a nonzero value if semaphore is unavailable
<code>void up(struct semaphore *sem)</code>	Releases the given semaphore
Reader-Writer Semaphores	
<code>void init_rwsem(struct rw_semaphore, *rwsem)</code>	Initializes the dynamically created semaphore with a count of 1
<code>void down_read(struct rw_semaphore, *rwsem)</code>	Down operation for readers
<code>void up_read(struct rw_semaphore, *rwsem)</code>	Up operation for readers
<code>void down_write(struct rw_semaphore, *rwsem)</code>	Down operation for writers
<code>void up_write(struct rw_semaphore, *rwsem)</code>	Up operation for writers

Linux Kernel Concurrency Mechanisms

Table 6.6 Linux Memory Barrier Operations

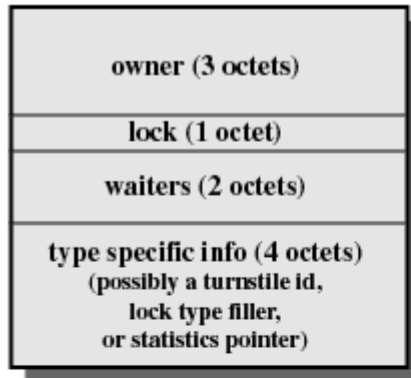
<code>rmb ()</code>	Prevents loads from being reordered across the barrier
<code>wmb ()</code>	Prevents stores from being reordered across the barrier
<code>mb ()</code>	Prevents loads and stores from being reordered across the barrier
<code>barrier ()</code>	Prevents the compiler from reordering loads or stores across the barrier
<code>smp_rmb ()</code>	On SMP, provides a <code>rmb ()</code> and on UP provides a <code>barrier ()</code>
<code>smp_wmb ()</code>	On SMP, provides a <code>wmb ()</code> and on UP provides a <code>barrier ()</code>
<code>smp_mb ()</code>	On SMP, provides a <code>mb ()</code> and on UP provides a <code>barrier ()</code>

SMP = symmetric multiprocessor

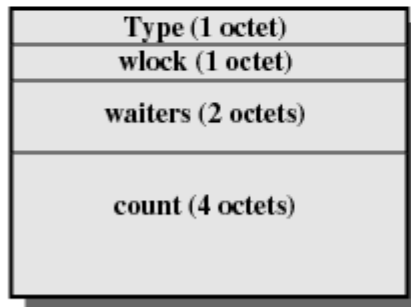
UP = uniprocessor

Solaris Thread Synchronization Primitives

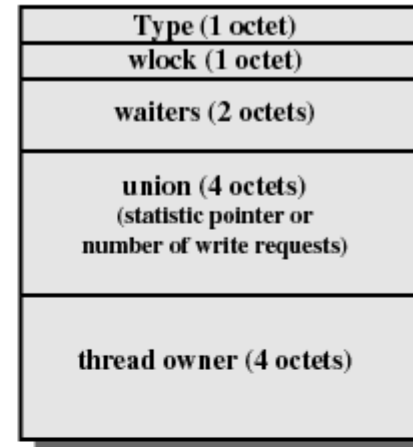
- Mutual exclusion (mutex) locks
- Semaphores
- Multiple readers, single writer
(readers/writer) locks
- Condition variables



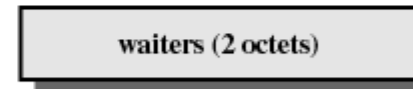
(a) MUTEX lock



(b) Semaphore



(c) Reader/writer lock



(d) Condition variable

Figure 6.15 Solaris Synchronization Data Structures

Table 6.7 Windows Synchronization Objects

Object Type	Definition	Set to Signaled State When	Effect on Waiting Threads
Event	An announcement that a system event has occurred	Thread sets the event	All released
Mutex	A mechanism that provides mutual exclusion capabilities; equivalent to a binary semaphore	Owning thread or other thread releases the mutex	One thread released
Semaphore	A counter that regulates the number of threads that can use a resource	Semaphore count drops to zero	All released
Waitable timer	A counter that records the passage of time	Set time arrives or time interval expires	All released
File change notification	A notification of any file system changes.	Change occurs in file system that matches filter criteria of this object	One thread released
Console input	A text window screen buffer (e.g., used to handle screen I/O for an MS-DOS application)	Input is available for processing	One thread released
Job	An instance of an opened file or I/O device	I/O operation completes	All released
Memory resource notification	A notification of change to a memory resource	Specified type of change occurs within physical memory	All released
Process	A program invocation, including the address space and resources required to run the program	Last thread terminates	All released
Thread	An executable entity within a process	Thread terminates	All released

Note: Colored rows correspond to objects that exist for the sole purpose of synchronization.